

Programmer-in-the-Middle Attacks on Flash Programming Equipment for Stealthy Firmware Implantation

Anonymous Author(s)

Abstract—TODO abstract to be written once results are finalized.

Index Terms—Firmware security, supply chain attacks, embedded systems, binary rewriting, hardware trust

I. INTRODUCTION

Software supply chain attacks increasingly target upstream dependencies rather than the build pipeline itself [1], [2] and static and LLM-based analysis tools are steadily raising the cost of injecting malicious code at compile time [3]. Direct compromise of the build process remains comparatively rare, but incidents such as SolarWinds and XZ Utils show it can evade the version-control and code-review mechanisms that dependency-poisoning defenses rely on [4]. These efforts, however, all target the build stage [5]. They say nothing about integrity once a binary leaves the build pipeline and must still be written onto physical hardware — a stage this paper argues is comparably exposed and far less scrutinized.

For embedded devices, software analysis, protection, and distribution practices have lagged behind general-purpose software security. Much of this gap traces to a physical constraint that conventional software distribution does not share: embedded firmware depends on physically flashing a device. The equipment used to perform this flashing is rarely subject to the same scrutiny as the firmware it programs. Cost pressure pushes smaller firms toward cloned or aftermarket programming hardware, and hardware assurance frameworks generally assume such equipment is trustworthy rather than verifying it. Guidance such as NIST IR 8259A and ETSI EN 303 645 scopes its provisions to the deployed device and its network exposure, leaving the programming stage and the equipment used to carry it out outside the trust boundary [6], [7].

STMicroelectronics’ official ST-Link programmers typically retail for USD 22–75, depending on the model. In contrast, compatible ST-Link V2 clones are widely available on marketplaces such as AliExpress and eBay for approximately USD 1.50–4. Many expose the same USB interface and SWD functionality expected by STM32CubeProgrammer and OpenOCD, allowing them to serve as drop-in replacements in standard flashing workflows [8].

The same transparency holds at the target end: clone MCUs on commercial Blue Pill boards present identical chip identifiers and debug interfaces to standard toolchains [9], [10], and counterfeit microprocessor and microcontroller ICs alone

accounted for 18 percent of all reported counterfeit component cases in a nine year high in 2024 [11]. At mass production scale, OEMs commonly outsource pre programming to third party providers who rely primarily on flash and readback verification as their core integrity check [12], [13], an assumption this work shows is straightforward to defeat from a compromised programmer or a malicious insider.

This gap between physical programming equipment and the scrutiny applied to firmware itself is the gap our work occupies. We propose programmer in the middle (PITM), an attack surface one level below the build pipeline and the source repository. By compromising the equipment used to flash firmware, an attacker can modify object code in place during the programming stage itself, after static analysis and code review have already passed, without altering anything in version control or in the build artifact handed to the programmer. We demonstrate this on three programmer architectures spanning distinct vendor toolchains: the ST-Link probe used across the full STM32 family, the onboard eZ-FET debugger on Texas Instruments MSP430 microcontrollers, and the USB-to-serial bridge controller used to flash classic AVR boards. Any tool of this kind that is a separate, programmable, and therefore independently compromisable microcontroller sitting between the host and the target device is exposed to the same attack.

We patch object code in place through trampoline-based rewriting that scans incoming object code for modification points directly, without recovering the target binary’s control-flow graph. Prior binary-rewriting approaches typically rely on this kind of control-flow or IR reconstruction [14], [15], which is too heavyweight for a compromised programmer operating on a single-pass data stream in real time. (further discussed in Section II). This supports attacks of increasing sophistication, from IVT hijacking to integrity check defeat and program extension or obstruction. The programmer defeats checksum and readback verification by scanning for patterns such as CRC comparison checks and conditional branches, patching the comparison to always succeed, then returning the original firmware instead of the modified write when the host reads the flash back to verify it.

This attack is particularly applicable to low-cost consumer devices such as IoT products, consumer electronics, and connected toys, where security assurance processes are generally less stringent than in safety-critical sectors and manufacturers often face strong cost and time-to-market pressures that limit deployment of hardware roots of trust, secure boot, and

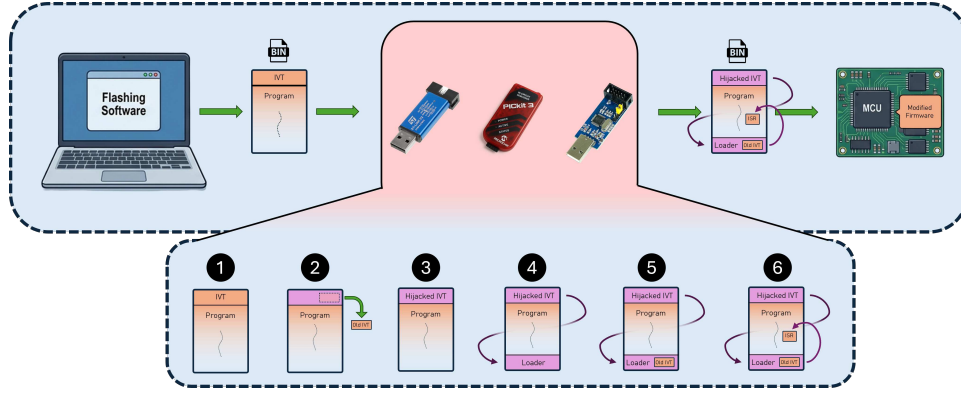


Fig. 1. Program Time Substitution

authenticated firmware update mechanisms [16]–[18]. Because the programmer’s readback remains clean, arbitrary firmware payloads can remain dormant until attacker-defined trigger conditions are met, for example to perform covert data exfiltration or intentionally degrade device functionality through mechanisms such as a hidden boot counter.

A compromised tool lying about what it wrote is not new. Stuxnet, BadUSB, and FLAW3D’s bootloader-resident implant each establish some version of the write-then-lie pattern [19]–[22]. PITM differs not in the pattern itself but in where it runs. It executes inside a separate, reusable tool already sitting at the lowest-level programming interface the target device exposes, leaving no lower-level interface through which the same programmer’s output can be independently validated. Mitigations therefore require an external trust anchor, such as an independently sourced programmer, out-of-band cryptographic attestation, physical or tamper-evident verification, side-channel analysis of the programmer, or independent electrical or optical inspection of the programmed device. The challenge is therefore fundamentally one of establishing an independent root of trust in the programming supply chain rather than modifying the programming protocol itself. In essence, PITM is Thompson’s trusting-trust argument [23] relocated to the programmer itself, the tool you must trust to tell you the truth about what it wrote.

The contributions of this paper are as follows.

- 1 We introduce **programmer in the middle (PITM)**, a new attack surface below the build pipeline and source repository, in which the flash programming equipment itself, rather than the firmware’s source, dependencies, or build artifacts, is the point of compromise.
- 2 We design and **demonstrate an in place object code modification technique**, performed through real time binary scanning rather than full structural or intermediate representation recovery.
- 3 We show this technique **operates within the memory constraints of the programming MCU itself**, an already resource constrained device, **without disturbing the timing tolerances** of a normal programming cycle on the host machine.
- 4 We show how this **technique defeats both checksum and post programming readback verification** through pattern based fingerprinting of integrity checks and bidirectional

spoofing of flash read operations.

5 We **catalog ten attack classes** enabled by a compromised programmer, ranging from IVT hijacking to program extension and obstruction, and argue this set can be extended through further tailoring.

6 We **deploy and evaluate our attack across the three programmer families with the most widely deployed microcontroller ecosystems**, demonstrating that the proposed technique is practical across diverse hardware and firmware implementations.

II. RELATED WORK

A. Software Supply Chain Attacks

Existing empirical and taxonomic work on software supply chain attacks consistently finds that dependency level compromise, including typosquatting, account takeover, and dependency confusion, dominates observed incidents, with direct compromise of the build pipeline or source code forming a smaller but disproportionately impactful minority [1], [2], [4]. These taxonomies are confined to conventional software supply chains. None model a stage analogous to physical device programming, where an artifact that has already passed every upstream check is modified after leaving the build pipeline entirely. PITM occupies this previously unmodeled stage.

B. Binary Rewriting and In-Place Code Modification

Binary rewriting, altering a compiled program’s behavior without access to its source, has been studied for over five decades, motivated variously by emulation, optimization, observation, and security hardening [14]. The most capable rewriting schemes recover a program’s control flow graph or lift it into an intermediate representation before transformation, which permits arbitrary instruction granularity edits but requires computational resources and time that a programmer operating in line during flashing does not have. We instead build on minimally invasive, trampoline based rewriting techniques that avoid control flow recovery entirely [15], redirecting execution at a small number of scanned, known pattern instrumentation points while leaving the surrounding binary, and its static jump targets, untouched.

C. Compromise of Programming and Debug Tools

A trusted tool lying about its own write during verification has documented precedent across several domains, and each instance must be positioned against carefully rather than dismissed with an unqualified novelty claim.

1) *Foundational Precedents: Stuxnet, BadUSB, and Trusting Trust*: Stuxnet’s compromised communications library performed both halves of the pattern this paper investigates: it injected malicious code into Siemens PLC program blocks during a download, and it filtered those same blocks out of any later read request from the engineering software, so the tool used to inspect the PLC’s programming displayed only what was legitimately present [19], [20]. BadUSB established the underlying principle in a different setting, a device’s firmware is the only channel through which that firmware can be verified, and a compromised device can simply lie along that channel [21]. Its demonstrated attacks, however, worked by having a USB controller impersonate an unauthorized device class, gaining only the capabilities of the class it posed as, rather than faithfully performing its declared role while covertly doing more. Thompson’s foundational argument, that a compromised tool can insert a backdoor invisible to inspection of the artifact it produces, is the conceptual ancestor of both [23]. Neither Stuxnet’s host-software compromise nor BadUSB’s device-class impersonation matches the specific configuration this paper targets, a hardware programmer that retains full, correct functionality while covertly patching and lying about an exact write, but both establish that the general write-then-lie pattern is not itself novel, and PITM’s contribution must rest on where the compromise sits rather than the pattern alone.

2) *FLAW3D: Bootloader-Resident Compromise*: FLAW3D is the closest prior instance to PITM and is treated separately here rather than folded into the discussion above. It plants a Trojan in the bootloader of AVR-based, Marlin-compatible 3D printers. The Trojan patches the stack-pointer initialization bytes of an incoming firmware image during the upload transaction, reclaiming RAM for its own state, then reverses that exact patch when the same bootloader performs the post-upload verification read [22]. Three properties separate it from PITM.

First, the two compromises differ in what a defender with independent access can actually find. FLAW3D’s Trojan occupies a bootloader stage that is architecturally separate from the application firmware, built on a standardized, open-source lineage shared across the more than 100 printer models the technique targets. ICSP reads that stage directly, bypassing the compromised bootloader, so a defender can diff it against a known-good reference and find the anomaly. PITM’s modification has no separate stage to isolate. It is fused into the application object code itself, a bespoke artifact that varies by target and by build, so there is no standardized reference to diff against, and the device class PITM targets offers no independent read access to attempt it.

Second, FLAW3D’s readback spoof hides one fixed, predetermined byte pattern at a known offset, produced deterministically by the target compiler’s own startup code. PITM’s defeat is pattern-matched against a general class of compare-and-

branch integrity check, generalizing to whatever was actually injected rather than one hardcoded diff decided in advance.

Third, and most significant for scope, FLAW3D’s bootloader-mediated protocol has a lower-level, hardware-enforced bypass on the same chip. AVR’s ICSP mode needs no bootloader code in the loop at all, and reading flash through it returns the true, unmediated contents. SWD/JTAG offers no equivalent fallback for the device class PITM targets. It is already the most direct programming interface available, which is precisely why a compromised tool operating at that level is a lower and more difficult trust boundary to defend.

3) *JTAG and Debug-Port Access Control*: A cluster of work addresses debug-port access control without engaging the failure mode PITM demonstrates, since each treats the programming tool as the trusted party being protected from external adversaries rather than as a potential adversary itself. Rosenfeld and Karri examine a malicious chip inserted into a JTAG scan chain, able to intercept data intended for a neighboring device and store it for later exfiltration, under the assumption that an attacker controls two chips flanking the target on an otherwise inactive bus [24]. Their own proposed mitigation, a star topology in place of a daisy chain, removes the shared-bus adjacency their attacks depend on to position a chip upstream or downstream of the victim, and the authors concede this directly: a star topology defeats many of the attacks they describe, though not every JTAG vulnerability. PITM requires no such adjacency to begin with. A host-to-target programming interface exists in star and chain topologies alike, and that interface, not a chain neighbor, is what PITM compromises. Park et al. propose a credential-based authentication scheme, gating JTAG access behind a hash of the chip’s secret key, a user password, and a permission level, with ordinary challenge-response sessions following successful authentication [25]. Valea et al. encrypt JTAG, IEEE 1500, and JTAG test traffic with a stream cipher to prevent a third party sharing the same scan chain from eavesdropping, an approach that explicitly assumes the tester itself is honest [26]. CHSM-Guard comes closest to naming our adversary directly, requiring a dedicated per-package hardware security module to authenticate an update tool via AEAD challenge-response before accepting any write, and explicitly listing compromised update tools among its threats [27]. Its defense, however, depends on security silicon absent from the cost-sensitive device class PITM targets, and is scoped to in-field chiplet updates rather than manufacturing-time flashing. All four are related-work completeness gaps rather than novelty threats: none permits the trusted tool itself to be the adversary that PITM’s threat model assumes from the outset.

Even granting that Rosenfeld and Karri’s register-modification and false-response attacks could in principle be composed into a single write-and-verify defeat, the composition does not survive contact with JTAG’s actual constraints. Five problems separate it from PITM.

First, the composed attack still needs chain adjacency, and the authors’ own proposed mitigation removes exactly that. Replacing the daisy chain with a star topology eliminates the shared-bus positioning both attacks require, and the authors concede a star topology defeats many of the attacks they

describe. PITM never depended on chain position to begin with. A programming interface sits between host and target in star and chain topologies alike, and that interface is what PITM compromises, so there is no equivalent escape for it to lose.

Second, the composed attack assumes knowledge the mechanism itself cannot supply. Writing persistent memory rather than transient register state requires knowing the target's proprietary flash control sequence, including vendor-specific unlock and program instructions beyond the generic boundary-scan path Rosenfeld and Karri demonstrate.

Third, it must operate inside a clock budget it does not control. TCK is bussed and the inserted chip is a slave to it, so any injection has to fit inside the cycles the legitimate master drives. Seizing the clock outright to buy more room reintroduces the electrical contention their own experimental section identifies as detectable. That narrow, externally-dictated window leaves little room to shift in a complete firmware image rather than a handful of substituted bits.

Fourth, the inserted chip has no way to independently read back the canonical firmware to know what a convincing spoofed verification should even contain. A read requires its own Capture-DR cycle and its own state transition, which it cannot initiate without the same TMS and TCK seizure already flagged as detectable. Whatever data does legitimately pass through it moves on toward its destination in real time, observed but not retained for later comparison.

Fifth, all of this assumes JTAG is even the interface exposed to begin with, and it often is not. JTAG is a debug interface first and an IEEE 1532 programming interface second, enabled by a manufacturer only when the debug capability is actually needed rather than by default. SWD, AVR-ICSP and ST-Link-class interfaces are the narrower floor most single-MCU, cost-sensitive boards actually expose for programming, and neither presents a chain for this class of attack to exploit at all, regardless of topology.

Two inserted chips would therefore need to jointly clock a full firmware image in and out inside a window neither one controls, using instructions neither is assumed to know, while also fabricating a byte-exact readback of content neither has ever actually read, and only if the target happens to expose a JTAG chain in the first place. That it is a considerably harder problem than just a logical extension, and most of it disappears the moment the bus is star-wired or the board exposes only SWD.

PITM sidesteps all of this by inverting where the trust sits. It is not a third party contending for a shared bus, it is the sole party the operator already trusts to perform the write and confirm it, so the write and the spoofed verification are carried out by the same authority that legitimately held the plaintext the entire time, with no clock budget to steal, no proprietary instruction set to infer, no canonical copy it lacks, and no dependency on JTAG being the exposed interface at all.

4) *HARVEY: PLC Rootkit*: HARVEY patches PLC firmware, in one demonstrated variant through the JTAG interface, using the same branch-to-injected-code-and-back trampoline structure employed here [28]. Once resident, it intercepts and fabricates both control outputs and sensor

telemetry during normal operation, showing an operator readings consistent with safe process behavior while the payload runs.

HARVEY's authors state directly that a novel infection vector was outside the scope of their work. They assume the attacker already holds a firmware-modification foothold, delivered through a firmware update, an SD card, or JTAG, and spend their contribution on a physics-aware model that computes adversary-optimal control commands and matching fabricated sensor readings. PITM contributes the foothold itself, the compromised tool as delivery-and-verification authority, and is agnostic to the payload carried through it. The two papers solve different halves of the same problem. Two distinctions follow. HARVEY's trampoline runs inside live firmware, fabricating telemetry continuously for as long as the device operates, far more exposed to side-channel screening than a payload that fires once. PITM's spoof fires once, at the verification read, and leaves. HARVEY also exists once per infected device and needs reinstallation per unit, while PITM's logic lives in the tool, so one compromise reaches every unit it flashes and leaves no resident rootkit on the target at all.

5) *Embedded and Industrial Control Rootkits*: A separate cluster demonstrates compromise of embedded control devices through means other than a shared manufacturing tool. MalTag is a parasitic hardware implant soldered onto a PLC's circuit board, hijacking board-level buses to override I/O signals live, an evolution of the compromise from software into a bolt-on physical addition rather than compromise of the legitimate tool already present in the manufacturing workflow [29]. Tan et al. achieve privilege escalation into a microcontroller's bootstrap loader through source-level re-entrancy, growing the stack via nested interrupts until the stack pointer reaches bootloader memory [30]. No binary patching is involved, making this an escalation attack against existing trusted ROM code rather than a technique for modifying arbitrary object code in place.

D. Detection Methods and Their Limits

Classical hardware Trojan detection targets modifications introduced during chip design or fabrication, through logic testing, side channel analysis of the die, and optical inspection. None of these apply to PITM, since the attack never alters silicon.

A more direct threat comes from on device firmware fingerprinting through power or electromagnetic side channels. Prior work has shown that electromagnetic emanations can support contactless control flow monitoring of embedded devices without modifying them [31], and that power traces can nonintrusively reveal unauthorized code running on embedded medical devices [32]. Screening of this kind could in principle be added to a production line, and would not depend on the programmer's reported readback at all. Its practical reach is limited in three ways. Extending power side channel analysis beyond cryptographic applications to general firmware behavior still faces significant noise and generalization challenges in real deployments [33]. Detection also requires a measurable behavioral difference between genuine and tampered execution. A patch that adds only the two or three instructions used

in our comparison rewrite produces a power or electromagnetic deviation far below what process variation and ambient noise already introduce on commodity parts, leaving nothing for a trace based detector to key on. The payload can also delay its action behind a counter or a timer, decoupling whatever side effect it eventually has from the specific window a verifier happens to be monitoring. Finally, production screening typically runs a short standardized workload, so a payload gated behind a rare or specific runtime trigger may simply not engage during the first few test runs and leaves no signature to detect, the same logic that lets rare trigger hardware Trojans evade conventional test patterns. A continuously active payload, by contrast, is far more exposed to this class of defense than one that fires rarely.

Hardware enforced control flow integrity offers a more direct counter to attacks such as IVT hijacking specifically. CFI CaRE adds hardware supported call and return enforcement to commercial microcontrollers [34], and ARM has since shipped a comparable capability as PACBTI on newer Armv8.1-M cores. This protection class remains confined to the newest and more expensive cores. By revenue, 32-bit microcontrollers already account for just over half of the global market [35], yet PACBTI class hardware is present on only a small and recent subset of those parts. The cost sensitive 8-bit, 16-bit, and budget 32-bit segments that dominate toys, basic appliances, and consumer IoT by unit count remain almost entirely unprotected at the architectural level.

Finally, the cleanest conceptual counter to readback spoofing is for a device to attest its own firmware over a channel the programmer cannot intercept. This is a well studied idea in the remote attestation literature, but hardware based and hybrid schemes require specialized tamper resistant hardware that most already deployed, cost sensitive IoT devices lack, and retrofitting or replacing that hardware is rarely practical [36]. Self verification of firmware integrity is, in other words, structurally difficult to achieve on exactly the class of device this work targets.

III. THREAT MODEL

A. System Model

A flash programmer sits between the host that issues a firmware image and the target microcontroller that receives it. The same programmer also performs the verification step, reading the contents of flash memory back to the host once writing is complete. This means a single component handles both the write path and the read path, and is the only source of ground truth the host has about what was actually placed on the device. If that component is compromised, the binary it writes and the binary it reports back need not be the same binary at all, since nothing else in the pipeline independently observes the chip's actual contents.

B. Adversary Model

The adversary controls the programmer itself rather than the source repository, the build pipeline, or the silicon of the target device. Once in place, the programmer can scan incoming object code in real time and patch it in place

without recovering its full control flow graph or lifting it to an intermediate representation, the same scanning capability used elsewhere in this paper to defeat checksum and readback verification. This single capability is enough to produce a wide range of effects depending on what the inserted code does once it runs, from silently disabling a boot sequence, to leaking or corrupting data, to introducing timing jitter or driving a peripheral outside its safe operating envelope. The demonstrations in this paper focus on a subset of these effects to establish feasibility rather than enumerate every possible outcome. We distinguish two ways this capability reaches real devices.

In an insider scale compromise, an attacker gains access to a single trusted point in a controlled environment, a compromised individual at a contract manufacturer, a certified distributor, or a targeted developer, and substitutes or backdoors the programmer used there. Gaining this foothold is hard, the same difficulty assumed by most hardware Trojan literature for a comparable initial compromise, but the payoff is disproportionate to the effort. A single compromised programmer that reaches a major manufacturer's production line taints every unit that line subsequently flashes, so a narrow, hard to achieve point of access produces an outcome that scales with the manufacturer's entire shipment volume rather than with the attacker's effort.

In a distribution scale compromise, the attacker does not need insider access at all. Backdoored clone programmers are introduced into the same low cost channels that already sell legitimate clones at a fraction of the official price, requiring no targeting of any specific manufacturer or product. Because the attacker does not know in advance which firmware will pass through any given unit, the payload must identify a target at scan time rather than being hardcoded to one, reusing the same pattern matching capability already used to find and defeat integrity checks to instead recognize signatures of the functionality or libraries it is meant to affect. Once a match is found, activation follows the same logic as a slow, deniable degradation rather than an immediate failure. If the target hardware exposes a real time clock, the payload can trigger on an absolute or universal time value, allowing an attacker to coordinate or stagger activation across the entire infected population independent of how any individual unit is used. Where no real time clock is available, which is common on the cost sensitive parts this paper targets, the payload falls back to a boot or power cycle counter, which requires nothing more than a persistent value in flash and is available on essentially any target.

C. Trust Assumptions

The model rests on one foundational assumption already motivated in the introduction, that programming equipment is treated by manufacturers and developers as inherently genuine, with no independent verification of the tool itself, a gap reflected directly in the price and availability of clone programmers for widely used MCU families. Where such verification is attempted at a QA stage, our technique relies on obfuscating the compromised programmer's own modifications to reduce

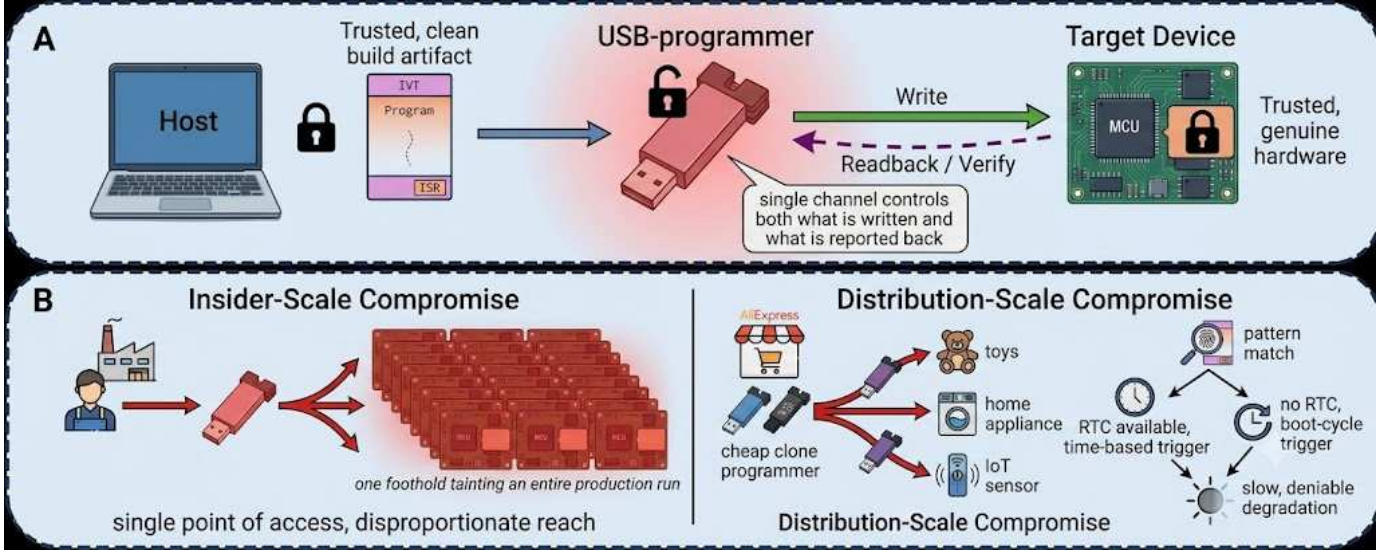


Fig. 2. Enter Caption

the likelihood of detection. Several conditions discussed in related work follow from this. There is no out of band channel through which a device attests its own firmware independently of the programmer that wrote it, since commercial MCUs in the segments this work targets do not implement that capability. Production lines do not deploy side channel screening capable of fingerprinting firmware behavior at scale, and where such screening exists in principle, the same code obfuscation circumvents pattern or signature based detection, in addition to the noise floor and trigger timing limitations discussed in related work. Target devices, being drawn from cost sensitive segments, do not implement hardware enforced control flow integrity. Finally, the firmware handed to the programmer is itself assumed clean, and the silicon of the target device is assumed genuine and unmodified, since compromise at either of those points is a different and already studied problem.

D. Scope

This model treats the programmer as the sole point of compromise and is agnostic to the broader supply chain that delivered a clean build artifact to it. We distinguish three classes of programming equipment under this model. Commercial programmers, the single unit development and production probes such as ST-Link and eZ-FET, are the class addressed throughout this paper. Gang programmers, used for higher throughput mass production flashing, fall into two further cases. Where a gang programmer is built as an ensemble of commercial probes operating in parallel, our model and demonstrated technique apply directly to each constituent unit, and the attack scales with the ensemble in the same way it would across any equivalent number of standalone commercial programmers. Production grade, fully proprietary gang programmers, which run closed implementations developed internally rather than assembled from commercial probes, are excluded from our evaluation, since their proprietary implementations place them outside what can be demonstrated here. The underlying model is nonetheless programmer agnostic

rather than specific to any one tool's implementation, so the same logic still applies to this class, it is simply harder to access and verify against. Fabrication level hardware Trojans on the programmer are also out of scope, since they require physical modification of silicon that this attack never performs.

IV. ATTACK CLASSES

TODO: catalog the ten attack classes referenced in Contribution 4, ordered roughly by increasing recalculation burden as framed in the Introduction. At minimum: IVT hijacking (redirect a vector table entry to a code-cave payload, no static jump target elsewhere is disturbed), control-flow-guided patching (branch-to-cave-and-back at a scanned instrumentation point without full CFG recovery), integrity-check defeat (pattern-match compare-and-branch sequences and force success), covert data exfiltration (e.g. GPIO toggling timed to a peripheral already active), planned obsolescence (boot/power-cycle counter gating a later failure), and program extension/obstruction (flagged in the Introduction as the hard case requiring PC-relative reference recalculation, out of scope for the current demonstration). Each class should state what it needs from the programmer (cave size, whether it must survive a reflash, whether it needs the RTC/boot-counter trigger from the Adversary Model) so the taxonomy ties back to Section III cleanly.

V. IMPLEMENTATION

A. ST-Link Firmware Analysis

TODO: describe the Ghidra-based reverse engineering of the ST-Link V2 firmware, correct base address (0x08004000, bootloader occupies the first 16KB), reset vector and startup chain, and identification of the target descriptor struct holding flash-algorithm pointers. Note the code-cave survey and the verification step distinguishing genuine dead space from .bss regions overwritten at runtime by CRT init.

B. Hook and Trampoline Mechanism

TODO: describe redirecting the flash-write function via the algorithm pointers identified above into the shellcode cave, the interception logic executed there, and the tail-call back into the original write routine so normal programmer operation is preserved.

C. Integrity Check Fingerprinting and Readback Spoofing

TODO: describe the pattern-matching pass that locates compare-and-branch integrity checks (e.g. CRC-then-CMP) in the incoming object code and neutralizes them, and the bidirectional spoofing of flash-read requests during the verification pass, tying back to the general-pattern-versus-fixed-pattern distinction drawn against FLAW3D in Related Work.

VI. EVALUATION

A. Experimental Setup

TODO: describe the hardware (ST-Link V2, genuine and clone, STM32F103 "Blue Pill/Neucleo" target), host tooling (STM32CubeProgrammer, OpenOCD), and the firmware sample set used, ideally spanning multiple compiler configurations as flagged in earlier planning, not a single fixed binary.

Also show a parallel USBasp and atmega328p as ISP programmer

B. Attack Success and Verification Evasion

TODO: report whether the IVT-hijack demonstration succeeds end to end, whether the write-then-verify workflow reports a clean match despite the modification, and any failure cases encountered. This is the empirical validation of the integrity-check-defeat claim that the novelty argument in the Introduction depends on; flag explicitly if any part of it remains unvalidated at submission time.

C. Generalization Beyond ST-Link

TODO: report the status of the TI eZ-FET and Arduino USB-serial bridge generalization claimed in the Introduction, full demonstration, partial, or architectural argument only.

D. Programmer Behavior Analysis

TODO: Export traces and power emi and behaviour and timing analysis of different compromised programmers. We have already established that the technique when not in progress has negligible impact on the target mcus behaviour especially with low flying atp style threats that trigger rarely but it remains important to catalogue the differences in timing power etc of the programmer itself as an enabler of this attack. Would QA and QC be able to narrow down to this fast enough, also introduce obfuscation to prevent easy detection from power traces

VII. DISCUSSION

A. Limitations

TODO: cover the Thumb-2 constraints on program extension (variable-length instructions, IT blocks, PC-relative literal pools, short branch range) that motivated restricting the demonstration to IVT hijacking, and the gang-programmer scoping already stated in Section III-D (ensemble gang programmers in scope, fully proprietary gang programmers excluded).

B. Defenses

TODO: summarize the countermeasure landscape already argued in Related Work and Trust Assumptions, side-channel screening, hardware CFI, remote attestation, cryptographic signing with a hardware root of trust, and state plainly which of these would actually stop PITM versus which only raise its cost.

VIII. CONCLUSION

TODO: restate the five contributions against what Sections IV–VII actually delivered, and close with the future-work items already identified: generalization to fully proprietary gang programmers, and out-of-band attestation as the countermeasure boundary.

REFERENCES

- [1] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. Springer, 2020, pp. 23–43.
- [2] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "Sok: Taxonomy of attacks on open-source software supply chains," in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 1509–1526.
- [3] D. Koterba, M. Łukaczyk, and W. Książek, "Leveraging large language models for advanced static code analysis: Assessing the feasibility of ai superseding traditional vulnerability detection methods," *Information and Software Technology*, vol. 198, p. 108213, 2026.
- [4] M. T. Diao, M. A. Diouf, I. E. Olatunji, A. K. Kaboré, G. Mendy, J. Klein, and T. F. Bissyandé, "Characterizing build compromises through vulnerability disclosure analysis," 2025.
- [5] E. T. A. M. S. G. Ifigeneia Lella, Marianthi Theocharidou, "Threat landscape for supply chain attacks," European Union Agency for Cybersecurity (ENISA), Tech. Rep., Jul. 2021. [Online]. Available: <https://www.enisa.europa.eu/sites/default/files/publications/ENISA%20Threat%20Landscape%20for%20Supply%20Chain%20Attacks.pdf>
- [6] M. Fagan, K. N. Megas, K. Scarfone, and M. Smith, "IoT device cybersecurity capability core baseline," National Institute of Standards and Technology, Tech. Rep. NIST IR 8259A, May 2020. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ir/2020/NIST.IR.8259A.pdf>
- [7] ETSI, "Cyber security for consumer internet of things: Baseline requirements," European Telecommunications Standards Institute, Tech. Rep. ETSI EN 303 645 V2.1.1, Jun. 2020. [Online]. Available: https://www.etsi.org/deliver/etsi_en/303600_303699/303645/02.01.01_60/en_303645v020101p.pdf
- [8] STMicroelectronics Community, "Fake ST-Links reaching me through 'legit' distributors: Which STCubeProgrammer version shall we use?" STMicroelectronics Community Forum, Jun. 2023, accessed June 2026. [Online]. Available: <https://community.st.com/t5/stm32-mcus-boards-and-hardware/fake-stlinks-reaching-me-through-quot-legit-quot-distributors/td-p/109895>
- [9] J. Obermaier, M. Schink, and K. Moczek, "One exploit to rule them all? on the security of drop-in replacement and counterfeit microcontrollers," 2020, uSENIX WOOT 2020.

- [10] M. Posch, “Stm32 clones: The good, the bad and the ugly,” <https://hackaday.com/2020/10/22/stm32-clones-the-good-the-bad-and-the-ugly/>, 2020, hackaday, accessed 2026-06-22.
- [11] Supply Chain Connect, “2024 counterfeit electronic parts report from erai,” <https://www.supplychainconnect.com/counterfeit/article/55311316/2024-counterfeit-electronic-parts-report-from-erai>, accessed 2026-06-22.
- [12] EPS Global, “Outsourced device programming: A simplified manufacturing solution,” <https://epsprogramming.com/blog/outsourced-device-programming>, accessed: 2026-06-22.
- [13] Silicon Labs, “Production programming options for mcu devices,” <https://www.silabs.com/software-and-tools/mcu-programming-options>, accessed: 2026-06-22.
- [14] M. Wenzl, G. Merzdovnik, J. Ullrich, and E. Weippl, “From hack to elaborate technique—a survey on binary rewriting,” *ACM Computing Surveys*, vol. 52, no. 3, pp. 1–37, 2019.
- [15] G. J. Duck, X. Gao, and A. Roychoudhury, “Binary rewriting without control flow recovery,” in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2020, pp. 151–163.
- [16] Internet Society, “The economics of the security of consumer-grade iot products and services,” Internet Society, Tech. Rep., Nov. 2019. [Online]. Available: <https://www.internetsociety.org/resources/doc/2019/the-economics-of-the-security-of-consumer-grade-iot-products-and-services/>
- [17] K. Zandberg, J. Delsing, and P. H. J. Nardelli, “Secure firmware updates for constrained iot devices using a scalable trust anchor,” *IEEE Access*, vol. 7, pp. 156 236–156 248, 2019.
- [18] M. M. H. Mahmud *et al.*, “A survey on secure over-the-air firmware update mechanisms for internet of things devices,” *Internet of Things*, vol. 19, p. 100553, 2022.
- [19] N. Falliere, L. O. Murchu, and E. Chien, “W32.stuxnet dossier,” Symantec Security Response, Tech. Rep., Feb. 2011, version 1.4.
- [20] R. Langner, “To kill a centrifuge: A technical analysis of what stuxnet’s creators tried to achieve,” The Langner Group, Tech. Rep., Nov. 2013.
- [21] K. Nohl and J. Lell, “Badusb – on accessories that turn evil,” Black Hat USA, 2014.
- [22] H. Pearce, K. Yanamandra, N. Gupta, and R. Karri, “FLAW3D: A trojan-based cyber attack on the physical outcomes of additive manufacturing,” *IEEE/ASME Transactions on Mechatronics*, vol. 27, no. 6, pp. 5361–5370, 2022.
- [23] K. Thompson, “Reflections on trusting trust,” *Communications of the ACM*, vol. 27, no. 8, pp. 761–763, 1984.
- [24] K. Rosenfeld and R. Karri, “Attacks and defenses for JTAG,” *IEEE Design & Test of Computers*, vol. 27, no. 1, pp. 36–47, 2010.
- [25] K. Park, S. G. Yoo, T. Kim, and J. Kim, “JTAG security system based on credentials,” *Journal of Electronic Testing*, vol. 26, pp. 549–557, 2010.
- [26] E. Valea, M. Da Silva, M.-L. Flottes, G. Di Natale, and B. Rouzeyre, “Encryption-based secure JTAG,” in *2019 IEEE 22nd International Symposium on Design and Diagnostics of Electronic Circuits & Systems (DDECS)*, 2019, pp. 1–6.
- [27] G. I. Haidar, J. Zhou, M. Tehranipoor, and F. Farahmandi, “CHSM-Guard: Secure boot and in-field firmware updates for chiplet-based SiPs,” in *Proceedings of the Great Lakes Symposium on VLSI (GLSVLSI)*, 2026.
- [28] L. Garcia, F. Brasser, M. H. Cintuglu, A.-R. Sadeghi, O. A. Mohammed, and S. A. Zonouz, “Hey, my malware knows physics! attacking PLCs with physical model aware rootkit,” in *24th Annual Network and Distributed System Security Symposium (NDSS)*, 2017.
- [29] M. Zhang and S. Zonouz, “Control corruption without firmware infection: Stealthy supply chain attacks via PLC hardware implants (MalTag),” in *2024 ACM/IEEE 15th International Conference on Cyber-Physical Systems (ICCPs)*, 2024, pp. 247–258.
- [30] S. J. Tan, S. Bratus, and T. Goodspeed, “Interrupt-oriented bugdoor programming: A minimalist approach to bugdooring embedded systems firmware,” in *Proceedings of the 30th Annual Computer Security Applications Conference (ACSAC)*, 2014, pp. 116–125.
- [31] Y. Han, S. Etigowni, H. Liu, S. Zonouz, and A. Petropulu, “Watch me, but don’t touch me! contactless control flow monitoring via electromagnetic emanations,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1095–1108.
- [32] S. S. Clark, B. Ransford, A. Rahmati, S. Guineau, J. Sorber, W. Xu, and K. Fu, “Wattsupdoc: Power side channels to nonintrusively discover untargeted malware on embedded medical devices,” in *2013 USENIX Workshop on Health Information Technologies (HealthTech 13)*, 2013.
- [33] P. Narimani, M. Wang, U. Planta, and A. Abbasi, “Exploring power side-channel challenges in embedded systems security,” 2024.
- [34] T. Nyman, J.-E. Ekberg, L. Davi, and N. Asokan, “Cfi care: Hardware-supported call and return enforcement for commercial microcontrollers,” in *Proceedings of the 20th International Symposium on Research in Attacks, Intrusions, and Defenses (RAID)*, 2017, pp. 259–284.
- [35] Grand View Research, “Microcontroller market size and share, industry report, 2033,” <https://www.grandviewresearch.com/industry-analysis/microcontroller-market>, accessed 2026-06-22.
- [36] S. F. J. J. Ankergård, E. Dushku, and N. Dragoni, “State-of-the-art software-based remote attestation: Opportunities and open issues for internet of things,” *Sensors*, vol. 21, no. 5, p. 1598, 2021.